Mohammed VI Polytechnic University (UM6P)

College of Computing

Ben Guerir, Morocco

Web Privacy

Practical Work: Browser Fingerprinting

Mission: Follow Alice

Student: Malek Awlaqi

Professor: Imane Fouad

Date: 1 October 2026

Contents

1 Introduction and objectives **2**

2 Project setup **2**

3 Part 1: passive fingerprinting **2**

3.1 Why . . . 2

3.2 Implementation . . . 3

3.3 What each header reveals . . . 3

3.4 Observations . . . 4

4 Part 2: active fingerprinting **5**

4.1 Why . . . 5

4.2 Implementation . . . 5

4.3 Why each feature helps to distinguish browsers . . . 7

4.4 Results . . . 7

4.5 Effect of protection mechanisms . . . 9

4.6 Most effective protection and its limitations . . . 11

5 Part 3: typing behaviour **11**

5.1 Why . . . 11

5.2 Design . . . 11

5.3 Typing speed formulas . . . 11

5.4 Implementation . . . 12

5.5 Result . . . 15

6 Part 4: a single SHA-256 identifier **15**

6.1 Why . . . 15

6.2 Implementation . . . 15

6.3 Properties and limits . . . 17

7 Conclusion **17**

1 Introduction and objectives

In the previous practical work, Alice was tracked with cookies stored in her browser. Once she deletes her cookies and blocks them, that technique no longer works. Browser fingerprinting answers this problem: instead of *storing* an identifier in the browser, the tracker *measures* characteristics that the browser already exposes. Individually these characteristics (language, screen size, time zone, graphics rendering) are harmless, but their combination can be distinctive enough to recognize a browser again without writing anything to it.

The goal of this work is to build a small local fingerprinting prototype and to evaluate how well it identifies a browser. The objectives are:

- to identify the information exposed passively through HTTP requests;
- to collect additional features actively with JavaScript;
- to measure a behavioural feature (typing speed and corrections);
- to combine the features into a single SHA-256 identifier;
- to observe which features change under different browsers and protections.

All tests were carried out locally (`127.0.0.1`) on a machine and browsers owned by the author, in line with the investigator's note of the assignment.

2 Project setup

The application is a Flask server that serves one page and receives the collected features. The structure is:

```
fingerprintingLab/
|-- app.py
|-- templates/
|   |-- index.html
|-- static/
    |-- fingerprint.js
    |-- fingerprint2.js
    |-- fingerprint3.js
```

A Python virtual environment is created and Flask is installed with `pip install flask`. The server runs on `http://127.0.0.1:8000`. Keeping the server local and the code transparent makes it possible to inspect every request in the browser developer tools and in the Flask console.

3 Part 1: passive fingerprinting

3.1 Why

Before a page is displayed, the browser sends an HTTP request whose headers describe the browser and its context. The server receives this information without running any code in the browser and without storing anything, which is why it is called *passive* collection. Studying it shows what any website learns the moment a user visits it.

3.2 Implementation

The route `/` of `app.py` (Listing 1) prints the client address, the HTTP method and several headers. Besides the `User-Agent` and `Accept-Language` headers given in the assignment, four additional headers were added: `Connection`, `Upgrade-Insecure-Requests`, `Accept-Encoding` and `Sec-Fetch-Site`.

Listing 1: app.py (passive collection and reception route)

```
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)


@app.route("/")
def home():
    print(" \n NEW VISIT  ")
    print("IP address : ", request.remote_addr)
    print(" HTTP methode :", request.method)
    print(" User agent :", request.headers.get("User-Agent"))
    print("Accept-Language :", request.headers.get("Accept-Language"))
    print("Connection          :", request.headers.get("Connection"))
    print("Upgrade-Insecure-Req:", request.headers.get("Upgrade-Insecure-Requests"))
    print("Accept-Encoding     :", request.headers.get("Accept-Encoding"))
    print("Sec-Fetch-Site      :", request.headers.get("Sec-Fetch-Site"))

    return render_template("index.html")


@app.route("/collect", methods=["POST"])
def collect():
    data = request.get_json(silent=True) or {}

    print("\n ACTIVE FEATURES RECEIVED")
    for key, value in data.items():
        print(f"{key:<26}: {value}")

    return jsonify({"status": "ok", "received": len(data)})


if __name__ == "__main__":
    app.run(host="127.0.0.1", port=8000, debug=True)
```

3.3 What each header reveals

Table 1: Passive HTTP features observed by the server.

Header	Observed value	Information revealed
IP address	127.0.0.1	Network address of the client; approximate location and provider in a real deployment.
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) ... Chrome/153.0.0.0	Operating system, architecture, browser family and version.
Accept-Language	en-US,en;q=0.9	Preferred languages and their priority, which reflects the user's configuration.
Accept-Encoding	gzip, deflate, br, zstd	Compression algorithms supported, which depends on the browser version.

Table 1: Passive HTTP features observed by the server.

Header	Observed value	Information revealed
Connection	keep-alive	How the browser manages connections.
Upgrade-Insecure-Requests	1	Indicates that the browser asks for upgrades to HTTPS.
Sec-Fetch-Site	none	Navigation context; `none` means a direct visit (typed address or bookmark).

3.4 Observations

Figures 1, 2 and 3 show the Flask console for three visits. The first visit (Figure 1) uses the first version of the collector, which only prints the user agent and the language; the later visits print the additional headers.

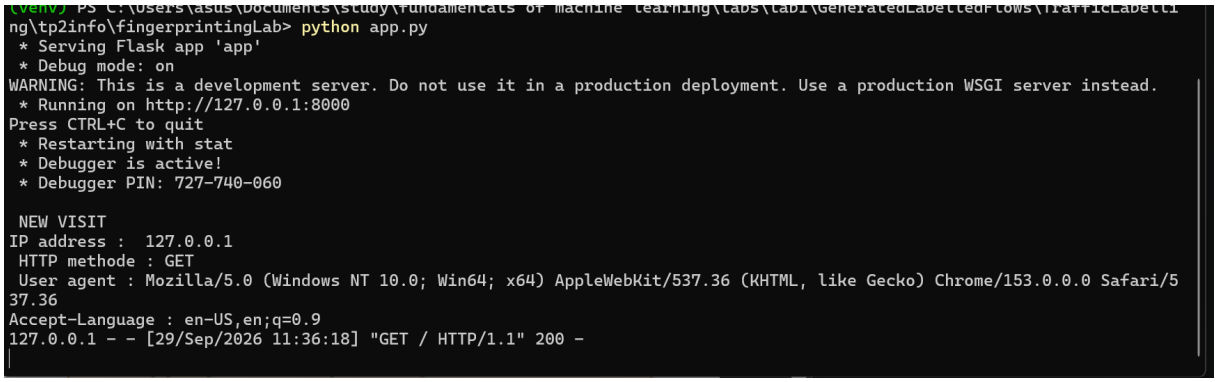


Figure 1: Flask console at the first visit (collector with two headers).

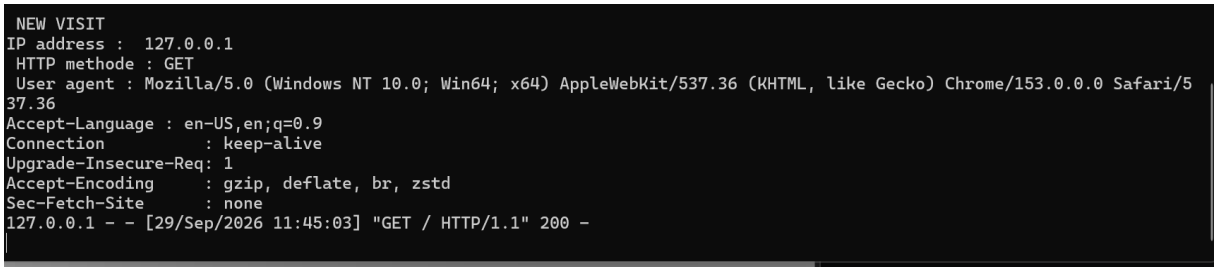


Figure 2: Flask console at 11:45:03 (collector with the additional headers).

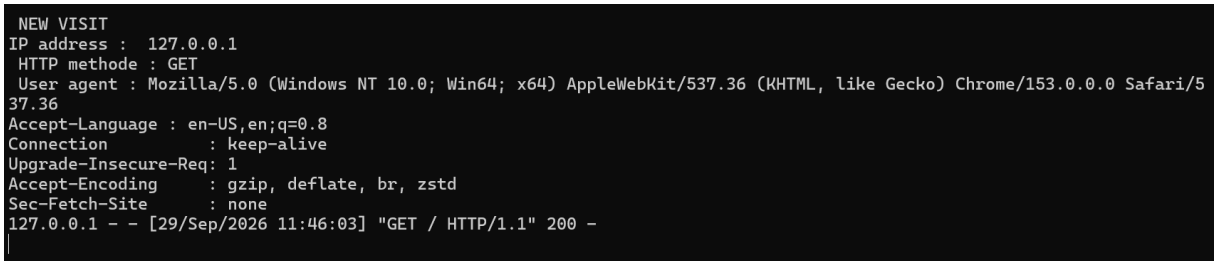


Figure 3: Flask console at 11:46:03.

Between the visits, the IP address, the user agent, `Accept-Encoding`, `Connection`, `Upgrade-Insecure-Requests` and `Sec-Fetch-Site` stayed identical, whereas the quality value

in `Accept-Language` was `q=0.9` at 11:45:03 and `q=0.8` at 11:46:03. This variation was observed but its cause was not investigated. The user agent also reads `Chrome/153.0.0.0` in the server log of 29 September and `Chrome/154.0.0.0` in the JavaScript output of 1 October, which shows that this header changes with the browser version.

4 Part 2: active fingerprinting

4.1 Why

Active fingerprinting executes JavaScript in the browser to query APIs whose values are not part of the HTTP request. Many of these properties (screen, hardware, rendering) are far more discriminating than headers, and most of them can be read without asking the user for any permission. This is why a tracker can still work after all cookies have been deleted.

4.2 Implementation

The script `fingerprint.js` (Listing 2) reads 19 features from the objects `navigator`, `screen`, `window`, `Intl` and `document`, including two graphics measurements (canvas and WebGL). The values are displayed in a table on the page and sent as JSON to the route `/collect`, where the Flask server prints them.

Listing 2: static/fingerprint.js

```
function getCanvasFingerprint() {
    try {
        const canvas = document.createElement("canvas");
        canvas.width = 220;
        canvas.height = 60;
        const ctx = canvas.getContext("2d");
        ctx.textBaseline = "top";
        ctx.font = "16px Arial";
        ctx.fillStyle = "#f60";
        ctx.fillRect(10, 5, 100, 30);
        ctx.fillStyle = "#069";
        ctx.fillText("Alice, cookies are gone :)", 4, 20);
        ctx.strokeStyle = "rgba(102, 204, 0, 0.7)";
        ctx.arc(150, 30, 20, 0, Math.PI * 2);
        ctx.stroke();
        const data = canvas.toDataURL();
        let h = 0;
        for (let i = 0; i < data.length; i++) {
            h = (h * 31 + data.charCodeAt(i)) | 0;
        }
        return "canvas-" + (h >>> 0).toString(16);
    } catch (e) {
        return "unavailable";
    }
}

function getWebGLRenderer() {
    try {
        const gl = document.createElement("canvas").getContext("webgl");
        if (!gl) return "unavailable";
        const ext = gl.getExtension("WEBGL_debug_renderer_info");
        if (!ext) return "masked";
        const vendor = gl.getParameter(ext.UNMASKED_VENDOR_WEBGL);
        return vendor + " | " + gl.getParameter(ext.UNMASKED_RENDERER_WEBGL);
    } catch (e) {
```

```
        return "unavailable";
    }
}

const features = {
    language: navigator.language,
    languages: (navigator.languages || []).join(","),
    userAgent: navigator.userAgent,
    platform: navigator.platform,
    hardwareConcurrency: navigator.hardwareConcurrency,
    deviceMemory: navigator.deviceMemory ?? "unavailable",
    cookieEnabled: navigator.cookieEnabled,
    doNotTrack: navigator.doNotTrack ?? "unavailable",

    screenResolution: screen.width + "x" + screen.height,
    availableScreen: screen.availWidth + "x" + screen.availHeight,
    colorDepth: screen.colorDepth,

    windowSize: window.innerWidth + "x" + window.innerHeight,
    devicePixelRatio: window.devicePixelRatio,

    timeZone: Intl.DateTimeFormat().resolvedOptions().timeZone,
    timezoneOffset: new Date().getTimezoneOffset(),

    referrer: document.referrer || "none",
    visibilityState: document.visibilityState,

    canvasFingerprint: getCanvasFingerprint(),
    webglRenderer: getWebGLRenderer(),
};

console.log("Collected features:", features);

const outputElement = document.getElementById("feature-output");
let html = "<table>";
for (const [key, value] of Object.entries(features)) {
    const safe = String(value).replace(/&/g, "&amp;").replace(/</g, "&lt;");
    html += `<tr><th>${key}</th><td>${safe}</td></tr>`;
}
html += "</table>";
outputElement.innerHTML = html;

fetch("/collect", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(features),
})
    .then((response) => response.json())
    .then((result) => {
        console.log("Server response:", result);
        document.getElementById("send-status").textContent =
            "Features sent to server (" + result.received + " received).";
    })
    .catch((error) => {
        console.error("Error sending features:", error);
        document.getElementById("send-status").textContent = "Failed to send features.";
    });
```

4.3 Why each feature helps to distinguish browsers

Table 2: Active features and what they reveal.

Feature	Object	What it reveals and why it distinguishes
language, languages	navigator	Interface language and the ordered list of accepted languages; long lists are rare and therefore discriminating.
userAgent, platform	navigator	Browser, version and operating system.
hardwareConcurrency	navigator	Number of logical processor cores; separates machine classes.
deviceMemory	navigator	Approximate memory in GB (rounded by the browser).
cookieEnabled, doNotTrack	navigator	Privacy settings chosen by the user.
screenResolution, availableScreen, colorDepth	screen	Display size, usable area and colour depth; depends on the monitor and scaling.
windowSize	window	Size of the browser window; very volatile, so it is weak over time.
devicePixelRatio	window	Display scaling (1.5 here), typical of a given screen and system setting.
timeZone, timezoneOffset	Intl, Date	Geographic region of the user.
referrer, visibilityState	document	Navigation context and page state.
canvasFingerprint	canvas	Hash of a rendered drawing; text and shape rasterization differ slightly between GPUs, drivers and fonts.
webglRenderer	WebGL	Vendor and name of the graphics processor, when the browser exposes it.

4.4 Results

Figures 4 and 5 show two sessions in Brave with the fingerprinting protection enabled, and Figure 6 shows Microsoft Edge without such protection.

Active Feature Collection

language	en-US
languages	en-US
userAgent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
platform	Win32
hardwareConcurrency	12
deviceMemory	4
cookieEnabled	true
doNotTrack	unavailable
screenResolution	1680x1050
availableScreen	1680x1050
colorDepth	24
windowSize	841x932
devicePixelRatio	1.5
timeZone	Africa/Casablanca
timezoneOffset	0
referrer	none
visibilityState	visible
canvasFingerprint	canvas-cc4fd3
webglRenderer	Brave | Brave

Figure 4: Active features collected in Brave, first session.

Active Feature Collection

language	en-US
languages	en-US
userAgent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
platform	Win32
hardwareConcurrency	22
deviceMemory	16
cookieEnabled	true
doNotTrack	unavailable
screenResolution	1680x1050
availableScreen	1680x1050
colorDepth	24
windowSize	842x932
devicePixelRatio	1.5
timeZone	Africa/Casablanca
timezoneOffset	0
referrer	none
visibilityState	visible
canvasFingerprint	canvas-be8650d3
webglRenderer	Brave | Brave

Figure 5: Active features collected in Brave, second session.

Active Feature Collection

language	fr
languages	fr,fr-FR,en,en-GB,en-US
userAgent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36 Edg/154.0.0.0
platform	Win32
hardwareConcurrency	32
deviceMemory	32
cookieEnabled	true
doNotTrack	unavailable
screenResolution	1707x1067
availableScreen	1707x1019
colorDepth	24
windowSize	1699x935
devicePixelRatio	1.5
timeZone	Africa/Casablanca
timezoneOffset	0
referrer	none
visibilityState	visible
canvasFingerprint	canvas-9c8ed41f
webglRenderer	Google Inc. (Intel) | ANGLE (Intel, Intel(R) UHD Graphics (0x0000A788) Direct3D11 vs_5_0 ps_5_0, D3D11)

Figure 6: Active features collected in Microsoft Edge.

4.5 Effect of protection mechanisms

Brave exposes the setting *Block fingerprinting* (Figures 7 and 8), together with options to block scripts and third-party cookies. The three runs above are compared in Table 3.

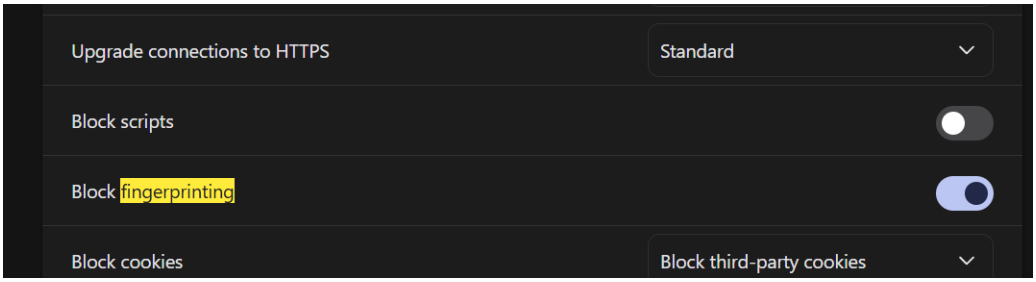


Figure 7: Brave settings: fingerprinting blocked, scripts not blocked, third-party cookies blocked.

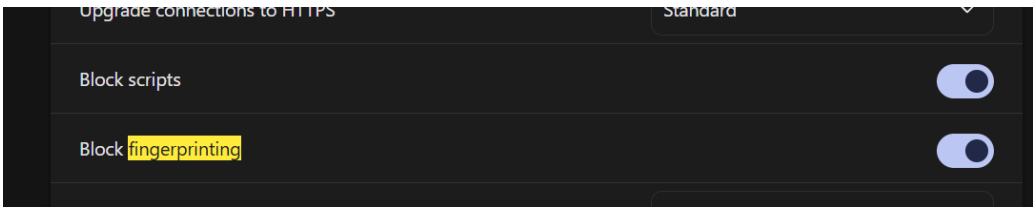


Figure 8: Brave settings: fingerprinting blocked and scripts blocked.

Feature	Brave, session 1	Brave, session 2	Edge
language	en-US	en-US	fr
languages	en-US	en-US	fr, fr-FR, en, en-GB, en-US
hardwareConcurrency	12	22	32
deviceMemory	4	16	32
screenResolution	1680x1050	1680x1050	1707x1067
windowSize	841x932	842x932	1699x935
canvasFingerprint	canvas-cc4fd3	canvas-be8650d3	canvas-9c8ed41f
webglRenderer	Brave | Brave	Brave | Brave	Google Inc. (Intel) | ANGLE (Intel, Intel(R) UHD Graphics, Direct3D11)

Table 3: Comparison of the features that differ between the three runs.

Features that stayed identical in all runs: `platform` (Win32), `colorDepth` (24), `devicePixelRatio` (1.5), `timeZone` (Africa/Casablanca), `timezoneOffset` (0), `cookieEnabled` (true), `doNotTrack` (unavailable), `referrer` (none) and `visibilityState` (visible).

Features that changed:

- `hardwareConcurrency` and `deviceMemory` took different values in the two Brave sessions (12 and 4, then 22 and 16), while Edge reports 32 and 32. The Brave values are therefore not stable from one session to another, which is consistent with randomization or reduction by the browser.
- `canvasFingerprint` also changed between the two Brave sessions, whereas a browser without protection would give the same hash on every reload.
- `webglRenderer` is replaced by the placeholder `Brave | Brave`, so the real graphics card is hidden. Edge exposes the exact model.
- `screenResolution` is 1680x1050 in Brave and 1707x1067 in Edge on the same display and with the same scaling ratio of 1.5, so Brave does not report the same screen size.
- `language` and `languages` differ between the browsers because of their own configuration; Edge exposes a longer and therefore more distinctive list.

Cookie blocking has no effect on this prototype because none of the collected features is read from or stored in a cookie: `cookieEnabled` still reports `true` and all other values are collected normally. Blocking scripts (Figure 8) gives the result of Figure 9: the page stays at *Waiting for JavaScript...* and no active feature is collected at all.



Figure 9: Result when scripts are blocked: no active feature is collected.

4.6 Most effective protection and its limitations

Among the protections tested, blocking scripts is the most effective against active fingerprinting, because the collector cannot run. Its limitation is that it breaks most modern websites and that passive information (IP address and HTTP headers, Part 1) is still sent. The fingerprinting protection of Brave keeps pages working and makes the values less reliable across sessions, but:

- several features remain stable (platform, colour depth, pixel ratio, time zone), and a tracker can still build a partial fingerprint from them;
- the placeholder values themselves (such as `Brave | Brave`) reveal which browser is used and that protection is active, which can make the user stand out within a smaller group;
- the protection does not apply to the typing behaviour of Part 3 or to the IP address.

Private-browsing mode and browser extensions were not part of the captured experiments, so no conclusion is drawn about them here. The comparison is also limited to one machine and two browsers.

5 Part 3: typing behaviour

5.1 Why

A fingerprint is not limited to device properties: the way a user interacts with a page can also be measured. Typing speed and the number of corrections characterize a person's rhythm and need no permission. The result is probabilistic, since fatigue, keyboard or posture change the behaviour, so it complements a device fingerprint without replacing it.

5.2 Design

Alice must type the fixed sentence *Cookies are gone, but Alice's rhythm still leaves traces!* The following algorithm was designed before writing the code:

1. read the target sentence, the input field, the button and the three result areas by their identifiers;
2. initialize `startTime` to empty, `corrections` to 0 and `finished` to false;
3. on `keydown`: if the key is Backspace, add 1 to `corrections`; otherwise, if it is a printable character and `startTime` is empty, store `performance.now()`;
4. on `input`: if the field content is strictly equal to the target sentence, store the end time, compute the results, display them, lock the field and send them to `/collect`;
5. block pasting, so that the sentence is really typed;
6. on `click` of the restart button: reset the state and the display.

5.3 Typing speed formulas

Let n be the number of characters of the sentence and T the total time in seconds between the first character and the exact match. The speed in characters per second is

$$v = \frac{n}{T}. \tag{1}$$

The speed in words per minute uses the usual convention of five characters per word:

$$\mathrm{WPM} = \frac{n/5}{T/60}. \tag{2}$$

Worked example. The sentence contains $n = 57$ characters (including spaces and punctuation). The measured time (Figure 10) is $T = 36.20$ s. Then

$$v = \frac{57}{36.20} = 1.5746\ldots \approx 1.57 \text{ chars/s}, \tag{3}$$

$$\mathrm{WPM} = \frac{57/5}{36.20/60} = \frac{11.4}{0.6033} = 18.89\ldots \approx 18.9, \tag{4}$$

which matches the values displayed by the page. Eleven Backspace presses were counted. Each correction adds time, so a high number of corrections lowers the speed.

5.4 Implementation

The page `index.html` (Listing 3) contains the sentence, the input field, the restart button and the result table, each with a unique identifier. The script `fingerprint2.js` (Listing 4) implements the algorithm above.

Listing 3: templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Browser Fingerprinting Lab</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 800px; margin: 40px auto; padding: 0 16px; }
        table { border-collapse: collapse; width: 100%; }
        th, td { border: 1px solid #ccc; padding: 6px 10px; text-align: left; word-break: break-all; }
        th { background: #f0f0f0; width: 30%; }
    </style>
</head>
<body>
    <h1>Mission: Follow Alice</h1>

    <h2>Active Feature Collection</h2>
    <p id="feature-output">Waiting for JavaScript...</p>
    <p id="send-status"></p>

    <h2>Browser Fingerprint (SHA-256)</h2>
    <p id="fp-output" style="font-family: monospace; word-break: break-all;">Computing...</p>
    <p id="fp-stability"></p>
    <p id="fp-status"></p>
    <details>
        <summary>Show the string that was hashed</summary>
        <p id="fp-string" style="font-family: monospace; word-break: break-all;"></p>
    </details>

    <h2>The Typing Imposter</h2>
    <p>Type this sentence exactly:</p>
    <p id="target-sentence"><strong>Cookies are gone, but Alice's rhythm still leaves traces!</strong></p>

```

```
    <input type="text" id="typing-input" size="60" autocomplete="off" spellcheck="false"
           placeholder="Start typing here...">
    <button id="restart-button">Restart</button>

    <table>
        <tr><th>Total typing time</th><td id="result-time"></td></tr>
        <tr><th>Typing speed</th><td id="result-speed"></td></tr>
        <tr><th>Corrections (Backspace)</th><td id="result-corrections"></td></tr>
    </table>
    <p id="typing-status"></p>

    <script src="{{ url_for('static', filename='fingerprint.js') }}"></script>
    <script src="{{ url_for('static', filename='fingerprint2.js') }}"></script>
    <script src="{{ url_for('static', filename='fingerprint3.js') }}"></script>
</body>
</html>
```

Listing 4: static/fingerprint2.js

```
const targetElement = document.getElementById("target-sentence");
const typingInput = document.getElementById("typing-input");
const restartButton = document.getElementById("restart-button");
const timeOutput = document.getElementById("result-time");
const speedOutput = document.getElementById("result-speed");
const correctionsOutput = document.getElementById("result-corrections");
const typingStatus = document.getElementById("typing-status");

const targetSentence = targetElement.textContent.trim();

let startTime = null;
let corrections = 0;
let finished = false;

typingInput.addEventListener("keydown", function (event) {
    if (finished) return;

    if (event.key === "Backspace") {
        corrections++;
        return;
    }

    if (startTime === null && event.key.length === 1) {
        startTime = performance.now();
    }
});

typingInput.addEventListener("input", function () {
    if (finished || startTime === null) return;

    if (typingInput.value === targetSentence) {
        finished = true;
        const endTime = performance.now();

        const totalSeconds = (endTime - startTime) / 1000;
        const charsPerSecond = targetSentence.length / totalSeconds;
        const wordsPerMinute = (targetSentence.length / 5) / (totalSeconds / 60);

        timeOutput.textContent = totalSeconds.toFixed(2) + " s";
        speedOutput.textContent =
            charsPerSecond.toFixed(2) + " chars/s (" + wordsPerMinute.toFixed(1) + " WPM)";
        correctionsOutput.textContent = corrections;

        typingInput.disabled = true;

```

```
        sendTypingFeatures({
            source: "typing",
            typingTimeSeconds: Number(totalSeconds.toFixed(2)),
            typingSpeedCharsPerSecond: Number(charsPerSecond.toFixed(2)),
            typingSpeedWPM: Number(wordsPerMinute.toFixed(1)),
            corrections: corrections,
        });
    }
});

typingInput.addEventListener("paste", function (event) {
    event.preventDefault();
});

restartButton.addEventListener("click", function () {
    startTime = null;
    corrections = 0;
    finished = false;

    typingInput.value = "";
    typingInput.disabled = false;
    timeOutput.textContent = "";
    speedOutput.textContent = "";
    correctionsOutput.textContent = "";
    typingStatus.textContent = "";
    typingInput.focus();
});

function sendTypingFeatures(data) {
    console.log("Typing features:", data);

    fetch("/collect", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(data),
    })
        .then((response) => response.json())
        .then((result) => {
            console.log("Server response:", result);
            typingStatus.textContent = "Typing features sent to server.";
        })
        .catch((error) => {
            console.error("Error sending typing features:", error);
            typingStatus.textContent = "Failed to send typing features.";
        });
}
```

5.5 Result

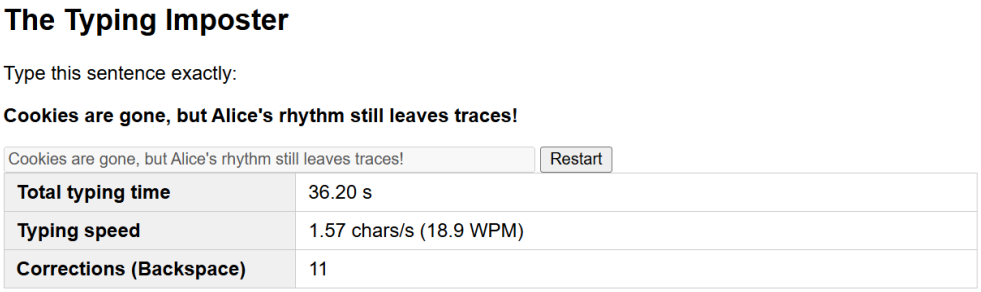


Figure 10: Result of the typing experiment: 36.20 s, 1.57 chars/s (18.9 WPM), 11 corrections.

With a single sample, this measurement cannot identify a person. A real system would gather many events over several sessions, including key-hold times and intervals between keys, before building a usable profile.

6 Part 4: a single SHA-256 identifier

6.1 Why

Separate features are inconvenient for a server to store and compare. Joining them in a fixed order and applying a hash function gives a fixed-length identifier. The same combination always produces the same value, the comparison is a simple string equality, and the raw features are no longer readable in the request, which is one reason fingerprinting is hard for an ordinary user to notice.

6.2 Implementation

The script `fingerprint3.js` (Listing 5) joins 15 features in a fixed order into one string of the form `name=value|name=value|...`, converts it to bytes and hashes it with `crypto.subtle.digest("SHA-256", ...)`. The result is written as 64 hexadecimal characters, displayed on the page and sent alone to the server. The function is executed five times to check that the hash is stable.

The behavioural features of Part 3 are excluded, as required, because they change at every attempt. The window size, the referrer and the visibility state are also excluded because they change from one page load to another and would make the identifier unstable without adding useful information.

Listing 5: static/fingerprint3.js

```
const STABLE_KEYS = [
    "language",
    "languages",
    "userAgent",
    "platform",
    "hardwareConcurrency",
    "deviceMemory",
    "cookieEnabled",
    "doNotTrack",
    "screenResolution",
```

```
    "availableScreen",
    "colorDepth",
    "devicePixelRatio",
    "timeZone",
    "canvasFingerprint",
    "webglRenderer",
];

function buildFingerprintString() {
    return STABLE_KEYS.map((key) => key + "=" + features[key]).join("|");
}

async function sha256(text) {
    const bytes = new TextEncoder().encode(text);
    const digest = await crypto.subtle.digest("SHA-256", bytes);
    return Array.from(new Uint8Array(digest))
        .map((b) => b.toString(16).padStart(2, "0"))
        .join("");
}

async function runFingerprint() {
    const fpOutput = document.getElementById("fp-output");
    const fpString = document.getElementById("fp-string");
    const fpStability = document.getElementById("fp-stability");
    const fpStatus = document.getElementById("fp-status");

    if (typeof features === "undefined") {
        fpOutput.textContent = "fingerprint.js did not run: no features available.";
        return;
    }
    if (!window.crypto || !crypto.subtle) {
        fpOutput.textContent = "crypto.subtle is unavailable (needs localhost or HTTPS).";
        return;
    }

    const raw = buildFingerprintString();
    const hash = await sha256(raw);

    fpOutput.textContent = hash;
    fpString.textContent = raw;
    console.log("Fingerprint string:", raw);
    console.log("Fingerprint (SHA-256):", hash);

    const hashes = [];
    for (let i = 0; i < 5; i++) {
        hashes.push(await sha256(buildFingerprintString()));
    }
    const stable = hashes.every((h) => h === hash);
    fpStability.textContent = stable
        ? "Stable: 5/5 generations gave the same hash."
        : "UNSTABLE: the hash changed between generations.";

    try {
        const response = await fetch("/collect", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ source: "fingerprint", fp: hash }),
        });
        const result = await response.json();
        console.log("Server response:", result);
        fpStatus.textContent = "Only the hash was sent to the server.";
    } catch (error) {
        console.error("Error sending fingerprint:", error);
```

```
74          fpStatus.textContent = "Failed to send the fingerprint.";
75      }
76  }
77
78  runFingerprint();
```

6.3 Properties and limits

A hash does not anonymize the data: the value is deterministic, so it still identifies the browser and can be used to link visits. It is also fragile, since changing one character in any feature produces a completely different hash. This is what protections such as the randomization seen in Part 2 exploit: they alter one feature so that the whole identifier changes between sessions.

7 Conclusion

This work shows that a browser can be recognized without any cookie. The HTTP request alone discloses the address, the browser and the languages. JavaScript adds screen, hardware, time zone and graphics features, and typing behaviour adds a measurable human component. Combining the stable features with SHA-256 gives one compact identifier.

The comparison of Brave and Edge shows that protections act mainly by hiding or randomizing individual values, and that blocking scripts stops active collection altogether at the cost of usability. No tested protection removes every signal, since passive headers, stable properties and behaviour remain observable. Fingerprints are also probabilistic and must not be treated as proof of identity.